Introduction to Dynamic Programming

Workshop – Term 1, Week 8

Susan He, Austin Song, Matthew Gu

Outline

1 Housekeeping

2 We have to go deeper

3 On being not a goldfish

4 More dimensions???

5 Appendix and resources

Attendance!

Please scan the attendance form so you can get free food

VJudge For Workshop

If you're confident with the content for today, feel free to get started on problems!

Link: https://vjudge.net/contest/802029

Password: `cpmsoc`

Outline

1 Housekeeping

2 We have to go deeper

3 On being not a goldfish

4 More dimensions???

5 Appendix and resources

A Problem

Hallway Tiling

A Problem

Hallway Tiling

Austin has a hallway with dimensions $1 \times N$. He also has two types of tiles: red square tiles (1×1) and blue rectangular tiles (1×2). How many different ways can Austin tile the hallway?

A Problem

Hallway Tiling

Austin has a hallway with dimensions $1 \times N$. He also has two types of tiles: red square tiles (1×1) and blue rectangular tiles (1×2). How many different ways can Austin tile the hallway?

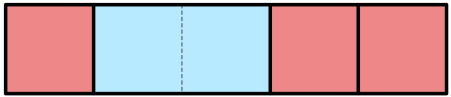
1. $N = 5$,
2. $N = 8$,
3. $1 \leq N \leq 10^9$.

A Problem

Hallway Tiling

Austin has a hallway with dimensions $1 \times N$. He also has two types of tiles: red square tiles (1×1) and blue rectangular tiles (1×2). How many different ways can Austin tile the hallway?

1. $N = 5$,
2. $N = 8$,
3. $1 \leq N \leq 10^9$.



N=5

Recursion

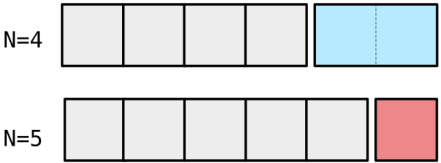
Let $A(n)$ be the solution for a 1 x n hallway. If $n > 2$ and we know the solution for $A(n-1)$ and $A(n-2)$, can we find $A(n)$?

Recursion

Let $A(n)$ be the solution for a 1 x n hallway. If $n > 2$ and we know the solution for $A(n-1)$ and $A(n-2)$, can we find $A(n)$?

We have two choices for the last tile:

- Blue: we have $A(n-2)$ ways of arranging all other tiles.
- Red: we have $A(n-1)$ ways of arranging all other tiles.

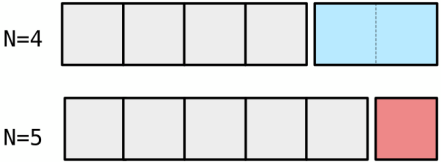


Recursion

Let $A(n)$ be the solution for a 1 x n hallway. If $n > 2$ and we know the solution for $A(n-1)$ and $A(n-2)$, can we find $A(n)$?

We have two choices for the last tile:

- Blue: we have $A(n-2)$ ways of arranging all other tiles.
- Red: we have $A(n-1)$ ways of arranging all other tiles.



Thus, we have the recurrence relation $A(n) = A(n-1) + A(n-2)$ for $n > 2$.

Recursion

We simplified the problem by breaking it down into subproblems.

To calculate $A(n)$, we need $A(n-1)$ and $A(n-2)$

- To calculate $A(n-1)$, we need $A(n-2)$ and $A(n-3)$
  - To calculate $A(n-2)$, we need $A(n-3)$ and $A(n-4)$
    - To calculate $A(n-3)$, we need $A(n-4)$ and $A(n-5)$
    - LaTeX won't let me add anymore
    - You get the point

When do we stop?

Recursion

We simplified the problem by breaking it down into subproblems.

To calculate $A(n)$, we need $A(n-1)$ and $A(n-2)$

- To calculate $A(n-1)$, we need $A(n-2)$ and $A(n-3)$
  - To calculate $A(n-2)$, we need $A(n-3)$ and $A(n-4)$
    - To calculate $A(n-3)$, we need $A(n-4)$ and $A(n-5)$
    - LaTeX won't let me add anymore
    - You get the point

When do we stop?

Base cases: $A(0)$ and $A(1)$. From this, we can generate all other $A(n)$.

Terminology

- Recursion: When a function calls/depends on itself.
- Base case: The simplest subproblem of the function. In our case, $N = 0, 1$.
- State: the variables that define a specific subproblem. This is the n in $A(n)$.
- State transitions: How states relate to each other, $A(n) = A(n-1) + A(n-2)$ for $n > 2$.

Implementation

Demo time!

Outline

1 Housekeeping

2 We have to go deeper

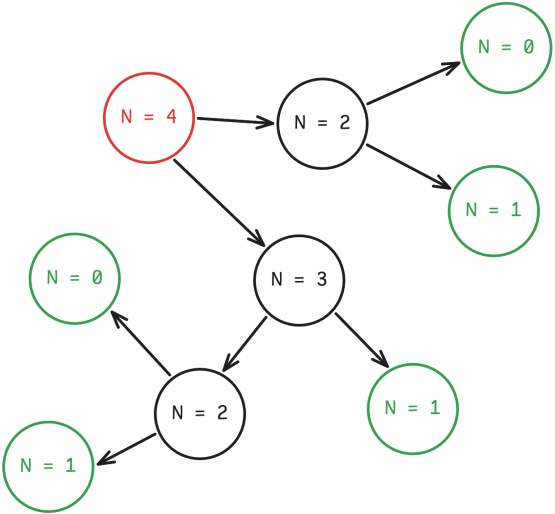
3 On being not a goldfish

4 More dimensions???

5 Appendix and resources

Back to Tiling Hallways!

Back to Tiling Hallways!



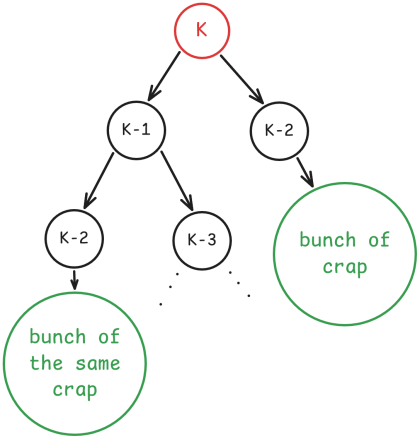
What's the obvious problem?

Why Is Our Program Stupid?

Exactly! Why are we recomputing $N = 2$ when we would've already done so? In general,

Why Is Our Program Stupid?

Exactly! Why are we recomputing $N = 2$ when we would've already done so? In general,



Solution: Remember

We can just **store** the solution for each state after we've solved for it!

Solution: Remember

We can just **store** the solution for each state after we've solved for it!

- We remember the answer for each state in our recursion.

Solution: Remember

We can just **store** the solution for each state after we've solved for it!

- We remember the answer for each state in our recursion.
- When we recurse onto a state, first ask ourselves:

Solution: Remember

We can just **store** the solution for each state after we've solved for it!

- We remember the answer for each state in our recursion.
- When we recurse onto a state, first ask ourselves:
  - Is this a base case which is trivially solvable?

Solution: Remember

We can just **store** the solution for each state after we've solved for it!

- We remember the answer for each state in our recursion.
- When we recurse onto a state, first ask ourselves:
  - Is this a base case which is trivially solvable?
  - If not, is this a case we've already solved?

Solution: Remember

We can just **store** the solution for each state after we've solved for it!

- We remember the answer for each state in our recursion.
- When we recurse onto a state, first ask ourselves:
  - Is this a base case which is trivially solvable?
  - If not, is this a case we've already solved?
- If yes, we can just recall the answer in $O(1)$.

Solution: Remember

We can just **store** the solution for each state after we've solved for it!

- We remember the answer for each state in our recursion.
- When we recurse onto a state, first ask ourselves:
  - Is this a base case which is trivially solvable?
  - If not, is this a case we've already solved?
- If yes, we can just recall the answer in $O(1)$.
- Else, we just keep recursing...

Solution: Remember

We can just **store** the solution for each state after we've solved for it!

- We remember the answer for each state in our recursion.
- When we recurse onto a state, first ask ourselves:
  - Is this a base case which is trivially solvable?
  - If not, is this a case we've already solved?
- If yes, we can just recall the answer in $O(1)$.
- Else, we just keep recursing...

Is this really that much better?

Solution: Remember

We can just **store** the solution for each state after we've solved for it!

- We remember the answer for each state in our recursion.
- When we recurse onto a state, first ask ourselves:
  - Is this a base case which is trivially solvable?
  - If not, is this a case we've already solved?
- If yes, we can just recall the answer in $O(1)$.
- Else, we just keep recursing...

Is this really that much better?

Let's find out by testing it – why not!

Time Complexity

It's a lot faster – but exactly how much faster?

Time Complexity

It's a lot faster – but exactly how much faster?

To analyse, consider the **total amount of work** that we do throughout the lifetime of the program.

Time Complexity

It's a lot faster – but exactly how much faster?

To analyse, consider the **total amount of work** that we do throughout the lifetime of the program.

- When we see a state for the first time, we perform real computation:

Time Complexity

It's a lot faster – but exactly how much faster?

To analyse, consider the **total amount of work** that we do throughout the lifetime of the program.

- When we see a state for the first time, we perform real computation:
  - In our case, it's grabbing $A(N-1)$ and $A(N-2)$; this is $O(1)$ from the perspective of $A(N)$.

Time Complexity

It's a lot faster – but exactly how much faster?

To analyse, consider the **total amount of work** that we do throughout the lifetime of the program.

- When we see a state for the first time, we perform real computation:
  - In our case, it's grabbing $A(N-1)$ and $A(N-2)$; this is $O(1)$ from the perspective of $A(N)$.
  - For more complex transitions, this may be $O(N)$ or $O(\log N)$ or whatever.

Time Complexity

It's a lot faster – but exactly how much faster?

To analyse, consider the **total amount of work** that we do throughout the lifetime of the program.

- When we see a state for the first time, we perform real computation:
  - In our case, it's grabbing $A(N-1)$ and $A(N-2)$; this is $O(1)$ from the perspective of $A(N)$.
  - For more complex transitions, this may be $O(N)$ or $O(\log N)$ or whatever.
- Every subsequent time we see a state, we don't bother – grab in $O(1)$.

Time Complexity

It's a lot faster – but exactly how much faster?

To analyse, consider the **total amount of work** that we do throughout the lifetime of the program.

- When we see a state for the first time, we perform real computation:
  - In our case, it's grabbing $A(N-1)$ and $A(N-2)$; this is $O(1)$ from the perspective of $A(N)$.
  - For more complex transitions, this may be $O(N)$ or $O(\log N)$ or whatever.
- Every subsequent time we see a state, we don't bother – grab in $O(1)$.
- Since we meaningfully 'visit' each state exactly once, our total time complexity is

$$O(\#\text{states} \cdot \text{transition time}).$$

Time Complexity

It's a lot faster – but exactly how much faster?

To analyse, consider the **total amount of work** that we do throughout the lifetime of the program.

- When we see a state for the first time, we perform real computation:
  - In our case, it's grabbing $A(N-1)$ and $A(N-2)$; this is $O(1)$ from the perspective of $A(N)$.
  - For more complex transitions, this may be $O(N)$ or $O(\log N)$ or whatever.
- Every subsequent time we see a state, we don't bother – grab in $O(1)$.
- Since we meaningfully 'visit' each state exactly once, our total time complexity is

$$O(\#\text{states} \cdot \text{transition time}).$$

For our Hallway Tiling problem, we've improved from $O(\text{idk}^N)$ to $O(N)$.

Time Complexity

It's a lot faster – but exactly how much faster?

To analyse, consider the **total amount of work** that we do throughout the lifetime of the program.

- When we see a state for the first time, we perform real computation:
  - In our case, it's grabbing $A(N-1)$ and $A(N-2)$; this is $O(1)$ from the perspective of $A(N)$.
  - For more complex transitions, this may be $O(N)$ or $O(\log N)$ or whatever.
- Every subsequent time we see a state, we don't bother – grab in $O(1)$.
- Since we meaningfully 'visit' each state exactly once, our total time complexity is

$$O(\#\text{states} \cdot \text{transition time}).$$

For our Hallway Tiling problem, we've improved from $O(\text{idk}^N)$ to $O(N)$.

Recursion combined with memory is known as **dynamic programming** (DP).

Bottom-Up DP

There's an easier way to see why the complexity improves:

Bottom-Up DP

There's an easier way to see why the complexity improves:

- Since we only visit each state once, let's try visiting them in a specific order.

Bottom-Up DP

There's an easier way to see why the complexity improves:

- Since we only visit each state once, let's try visiting them in a specific order.
- If a state x depends on y, let's solve y before x.
  - There should be no conflicts – else recursion wouldn't terminate anyway!
  - For the Hallway Tiling problem, this looks like visiting $0, 1, 2, 3, \ldots, N$.

Bottom-Up DP

There's an easier way to see why the complexity improves:

- Since we only visit each state once, let's try visiting them in a specific order.
- If a state x depends on y, let's solve y before x.
  - There should be no conflicts – else recursion wouldn't terminate anyway!
  - For the Hallway Tiling problem, this looks like visiting $0, 1, 2, 3, \ldots, N$.
- Then, we loop through N elements, and perform our transition at each one!

Bottom-Up DP

There's an easier way to see why the complexity improves:

- Since we only visit each state once, let's try visiting them in a specific order.
- If a state x depends on y, let's solve y before x.
  - There should be no conflicts – else recursion wouldn't terminate anyway!
  - For the Hallway Tiling problem, this looks like visiting $0, 1, 2, 3, \ldots, N$.
- Then, we loop through N elements, and perform our transition at each one!

This method of 'skipping the recursion' is known as **tabulation**, or **bottom-up** dynamic programming.

Let's implement it (for edification's sake).

Bottom-Up DP

There's an easier way to see why the complexity improves:

- Since we only visit each state once, let's try visiting them in a specific order.
- If a state x depends on y, let's solve y before x.
  - There should be no conflicts – else recursion wouldn't terminate anyway!
  - For the Hallway Tiling problem, this looks like visiting $0, 1, 2, 3, \ldots, N$.
- Then, we loop through N elements, and perform our transition at each one!

This method of 'skipping the recursion' is known as **tabulation**, or **bottom-up** dynamic programming.

Let's implement it (for edification's sake).

P.S. If you've done COMP2521, you'll recognise that recursive DP is basically DFS! Then, you don't need to think bottom-up to realise why the time complexity improves so much.

When Can We Do DP?

DP is a subset of recursion (recursion with extra steps), so we can use it when:

When Can We Do DP?

DP is a subset of recursion (recursion with extra steps), so we can use it when:

- We can use recursion – i.e.

When Can We Do DP?

DP is a subset of recursion (recursion with extra steps), so we can use it when:

- We can use recursion – i.e.
  - No weird cyclic dependencies,

When Can We Do DP?

DP is a subset of recursion (recursion with extra steps), so we can use it when:

- We can use recursion – i.e.
  - No weird cyclic dependencies,
  - Easy to define a state,

When Can We Do DP?

DP is a subset of recursion (recursion with extra steps), so we can use it when:

- We can use recursion – i.e.
  - No weird cyclic dependencies,
  - Easy to define a state,
  - There are nice recursive subproblems.

When Can We Do DP?

DP is a subset of recursion (recursion with extra steps), so we can use it when:

- We can use recursion – i.e.
  - No weird cyclic dependencies,
  - Easy to define a state,
  - There are nice recursive subproblems.
- Also, the state has to be easy to store (and reasonably sized)!
  - For instance, if our state was an entire array, it would be hard to DP over that...

When Can We Do DP?

DP is a subset of recursion (recursion with extra steps), so we can use it when:

- We can use recursion – i.e.
  - No weird cyclic dependencies,
  - Easy to define a state,
  - There are nice recursive subproblems.
- Also, the state has to be easy to store (and reasonably sized)!
  - For instance, if our state was an entire array, it would be hard to DP over that...

If these things are true, we can just chuck a `dp[state]` array over our recursion and steal the free optimisation.

Optimisation Problems

Froggy

Optimisation Problems

Froggy

There are N stones, labelled from 1 to N. The height of the i'th stone is h_i.

Optimisation Problems

Froggy

There are N stones, labelled from 1 to N. The height of the i'th stone is h_i.

Frorgy is initially on stone 1. It'll repeat the following action to reach N:

- If fooryg is on stone i, it'll hop to a stone $j \in [i+1, i+K]$ of its choice.

Optimisation Problems

Froggy

There are N stones, labelled from 1 to N. The height of the i'th stone is h_i.

Frorgy is initially on stone 1. It'll repeat the following action to reach N:

- If fooryg is on stone i, it'll hop to a stone $j \in [i+1, i+K]$ of its choice.

If frogry changes height too much, it'll get **annoyed**. It starts off not annoyed. The **annoyance** level increases by $|h_i - h_j|$ when jumping from i to j.

Optimisation Problems

Froggy

There are N stones, labelled from 1 to N. The height of the i'th stone is h_i.

Frorgy is initially on stone 1. It'll repeat the following action to reach N:

- If fooryg is on stone i, it'll hop to a stone $j \in [i+1, i+K]$ of its choice.

If frogry changes height too much, it'll get **annoyed**. It starts off not annoyed. The **annoyance** level increases by $|h_i - h_j|$ when jumping from i to j.

Find the minimum possible annoyance of rfogyg when it reaches N.

Optimisation Problems

Froggy

There are N stones, labelled from 1 to N. The height of the i'th stone is h_i.

Frorgy is initially on stone 1. It'll repeat the following action to reach N:

- If fooryg is on stone i, it'll hop to a stone $j \in [i+1, i+K]$ of its choice.

If frogry changes height too much, it'll get **annoyed**. It starts off not annoyed. The **annoyance** level increases by $|h_i - h_j|$ when jumping from i to j.

Find the minimum possible annoyance of rfogyg when it reaches N.

Constraints:

- $N \leq 10^5$, $K \leq 10^2$.

Froggy – Solution

The problem seems to be solvable recursively...

Froggy – Solution

The problem seems to be solvable recursively...

- Let's say froggy is at N. Then, it could've come from any of $x \in [N - K, N - 1]$.

Froggy – Solution

The problem seems to be solvable recursively...

- Let's say froggy is at N. Then, it could've come from any of $x \in [N - K, N - 1]$.
- We then have to find the minimum cost to get to any one of these x. This is basically the same as solving to get to N (except it's not N, obviously).

Froggy – Solution

The problem seems to be solvable recursively...

- Let's say froggy is at N. Then, it could've come from any of $x \in [N - K, N - 1]$.
- We then have to find the minimum cost to get to any one of these x. This is basically the same as solving to get to N (except it's not N, obviously).
- So, this motivates:

Froggy – Solution

The problem seems to be solvable recursively...

- Let's say froggy is at N. Then, it could've come from any of $x \in [N - K, N - 1]$.
- We then have to find the minimum cost to get to any one of these x. This is basically the same as solving to get to N (except it's not N, obviously).
- So, this motivates:
  1. **State**: $dp(x) :=$ min cost to get from 1 to x.

Froggy – Solution

The problem seems to be solvable recursively...

- Let's say froggy is at N. Then, it could've come from any of $x \in [N-K, N-1]$.
- We then have to find the minimum cost to get to any one of these x. This is basically the same as solving to get to N (except it's not N, obviously).
- So, this motivates:
  1. **State**: $dp(x) :=$ min cost to get from 1 to x.
  2. **Transition**:

$$dp(x) = \min_{i \in [x-K, x-1]} (dp(i) + |h_i - h_x|).$$

Froggy – Solution

The problem seems to be solvable recursively...

- Let's say froggy is at N. Then, it could've come from any of $x \in [N-K, N-1]$.
- We then have to find the minimum cost to get to any one of these x. This is basically the same as solving to get to N (except it's not N, obviously).
- So, this motivates:
  1. **State**: $dp(x) :=$ min cost to get from 1 to x.
  2. **Transition**:
  $$dp(x) = \min_{i \in [x-K, x-1]} (dp(i) + |h_i - h_x|).$$
  3. **Base Cases**: $dp(x) = 0$ for all $x \leq 1$.

Froggy – Solution

The problem seems to be solvable recursively...

- Let's say froggy is at N. Then, it could've come from any of $x \in [N-K, N-1]$.
- We then have to find the minimum cost to get to any one of these x. This is basically the same as solving to get to N (except it's not N, obviously).
- So, this motivates:
  1. **State**: $dp(x) :=$ min cost to get from 1 to x.
  2. **Transition**:
  $$dp(x) = \min_{i \in [x-K, x-1]} (dp(i) + |h_i - h_x|).$$
  3. **Base Cases**: $dp(x) = 0$ for all $x \leq 1$.
- Our answer is simply $dp(N)$.

Froggy – Solution

The problem seems to be solvable recursively...

- Let's say froggy is at N. Then, it could've come from any of $x \in [N-K, N-1]$.
- We then have to find the minimum cost to get to any one of these x. This is basically the same as solving to get to N (except it's not N, obviously).
- So, this motivates:
  1. **State**: $dp(x) :=$ min cost to get from 1 to x.
  2. **Transition**:
  $$dp(x) = \min_{i \in [x-K, x-1]} (dp(i) + |h_i - h_x|).$$
  3. **Base Cases**: $dp(x) = 0$ for all $x \leq 1$.
- Our answer is simply $dp(N)$.

Now that we have a recursive formulation, we just chuck DP over it and win :)

Exercise: What's the final time complexity?

Outline

1 Housekeeping

2 We have to go deeper

3 On being not a goldfish

4 More dimensions???

5 Appendix and resources

A Problem

Ain't No Party Like a DP Party!

A Problem

Ain't No Party Like a DP Party!

D. Piddy is hosting one of his **world-renowned** DP parties, where people of all ages come together to solve DP problems.

A Problem

Ain't No Party Like a DP Party!

D. Piddy is hosting one of his **world-renowned** DP parties, where people of all ages come together to solve DP problems.

The party is located in his mansion, which is in cell (R, C). You start in cell $(1, 1)$ and can only move down or right in each step.

A Problem

Ain't No Party Like a DP Party!

D. Piddy is hosting one of his **world-renowned** DP parties, where people of all ages come together to solve DP problems.

The party is located in his mansion, which is in cell (R, C). You start in cell $(1, 1)$ and can only move down or right in each step.

Unfortunately, D. Piddy is under investigation by the federal police.

A Problem

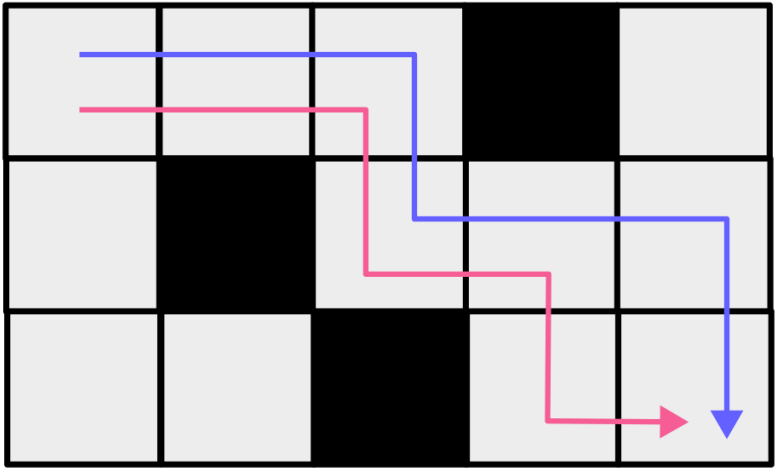
Ain't No Party Like a DP Party!

D. Piddy is hosting one of his **world-renowned** DP parties, where people of all ages come together to solve DP problems.

The party is located in his mansion, which is in cell (R, C). You start in cell $(1, 1)$ and can only move down or right in each step.

Unfortunately, D. Piddy is under investigation by the federal police. If $p_{i,j} = 1$, it means police are present at cell (i, j), and will arrest anyone they see going to the party. How many ways are there to reach Piddy's mansion, while avoiding police detection?

Visualisation



Solution

What do we need in our state?

Solution

What do we need in our state?

- The most natural way to reduce the problem is to consider the subproblem: how many ways to reach cell (i, j)?

Solution

What do we need in our state?

- The most natural way to reduce the problem is to consider the subproblem: how many ways to reach cell (i, j)?
- To make this our state, we must extend our DP into 2 dimensions.

Solution

What do we need in our state?

- The most natural way to reduce the problem is to consider the subproblem: how many ways to reach cell (i, j)?
- To make this our state, we must extend our DP into 2 dimensions.
- So, this motivates:

Solution

What do we need in our state?

- The most natural way to reduce the problem is to consider the subproblem: how many ways to reach cell (i, j)?
- To make this our state, we must extend our DP into 2 dimensions.
- So, this motivates:
  1. **State**: $dp(i, j) :=$ number of ways to get from $(1, 1)$ to cell (i, j).

Solution

What do we need in our state?

- The most natural way to reduce the problem is to consider the subproblem: how many ways to reach cell (i, j)?
- To make this our state, we must extend our DP into 2 dimensions.
- So, this motivates:
  1. **State**: $dp(i, j) :=$ number of ways to get from $(1, 1)$ to cell (i, j).
  2. **Transition**:

$$dp(i,j) = \begin{cases} 0 & \text{if } p_{ij} = 1 \\ dp(i-1,j) + dp(i,j-1) & \text{if } p_{i,j} = 0 \end{cases}$$

Solution

What do we need in our state?

- The most natural way to reduce the problem is to consider the subproblem: how many ways to reach cell (i, j)?
- To make this our state, we must extend our DP into 2 dimensions.
- So, this motivates:
  1. **State**: $dp(i, j) :=$ number of ways to get from $(1, 1)$ to cell (i, j).
  2. **Transition**:
  $$dp(i,j) = \begin{cases} 0 & \text{if } p_{ij} = 1 \\ dp(i-1,j) + dp(i,j-1) & \text{if } p_{i,j} = 0 \end{cases}$$
  3. **Base Cases**: $dp(1, 1) = 1$.

Solution

What do we need in our state?

- The most natural way to reduce the problem is to consider the subproblem: how many ways to reach cell (i, j)?
- To make this our state, we must extend our DP into 2 dimensions.
- So, this motivates:
  1. **State**: $dp(i, j) :=$ number of ways to get from $(1, 1)$ to cell (i, j).
  2. **Transition**:
  $$dp(i,j) = \begin{cases} 0 & \text{if } p_{ij} = 1 \\ dp(i-1,j) + dp(i,j-1) & \text{if } p_{i,j} = 0 \end{cases}$$
  3. **Base Cases**: $dp(1, 1) = 1$.
- Our answer is simply $dp(R, C)$.

Solution

What do we need in our state?

- The most natural way to reduce the problem is to consider the subproblem: how many ways to reach cell (i, j)?
- To make this our state, we must extend our DP into 2 dimensions.
- So, this motivates:
  1. **State**: $dp(i, j) :=$ number of ways to get from $(1, 1)$ to cell (i, j).
  2. **Transition**:
  $$dp(i,j) = \begin{cases} 0 & \text{if } p_{ij} = 1 \\ dp(i-1,j) + dp(i,j-1) & \text{if } p_{i,j} = 0 \end{cases}$$
  3. **Base Cases**: $dp(1, 1) = 1$.
- Our answer is simply $dp(R, C)$.

Exercise: What's the final time complexity?

Challenge Problem

Piddy's Problems

Challenge Problem

Piddy's Problems

Apart from the police, D. Piddy has another major issue. His DP party is tonight and he needs to pick problems for it!

Challenge Problem

Piddy's Problems

Apart from the police, D. Piddy has another major issue. His DP party is tonight and he needs to pick problems for it!

There are N DP problems, sorted by their unique difficulty $d_i <= D$. Piddy wants his problem set to cater to **all ages and skill levels**, so he needs to choose a set of problems which are equally spaced in difficulty.

Challenge Problem

Piddy's Problems

Apart from the police, D. Piddy has another major issue. His DP party is tonight and he needs to pick problems for it!

There are N DP problems, sorted by their unique difficulty $d_i <= D$. Piddy wants his problem set to cater to **all ages and skill levels**, so he needs to choose a set of problems which are equally spaced in difficulty.

Help him find the biggest possible size, under this restriction!

Challenge Problem

Piddy's Problems

Apart from the police, D. Piddy has another major issue. His DP party is tonight and he needs to pick problems for it!

There are N DP problems, sorted by their unique difficulty $d_i <= D$. Piddy wants his problem set to cater to **all ages and skill levels**, so he needs to choose a set of problems which are equally spaced in difficulty.

Help him find the biggest possible size, under this restriction!

Part I: $N = 1,000$, $D = 10,000$.

Challenge Problem

Piddy's Problems

Apart from the police, D. Piddy has another major issue. His DP party is tonight and he needs to pick problems for it!

There are N DP problems, sorted by their unique difficulty $d_i <= D$. Piddy wants his problem set to cater to **all ages and skill levels**, so he needs to choose a set of problems which are equally spaced in difficulty.

Help him find the biggest possible size, under this restriction!

Part I: $N = 1,000$, $D = 10,000$.

Part II: $N = 1,000$, $D = 10,000,000$.

Samples

Sample Cases

Sample Input:

```
8
1 3 4 7 9 10 11 14
```

Sample Output:

```
4
```

Part 1

What do we need in our state?

Part 1

What do we need in our state?

- The most natural state is $dp(i) :=$ max length of a problem set ending at i.

Part 1

What do we need in our state?

- The most natural state is $dp(i) :=$ max length of a problem set ending at i.
- But this is not enough information to transition – we need to know the spacing s between problems across states.

Part 1

What do we need in our state?

- The most natural state is $dp(i) :=$ max length of a problem set ending at i.
- But this is not enough information to transition – we need to know the spacing s between problems across states.
- So, this motivates:

Part 1

What do we need in our state?

- The most natural state is $dp(i) :=$ max length of a problem set ending at i.
- But this is not enough information to transition – we need to know the spacing s between problems across states.
- So, this motivates:
  1. **State**: $dp(i, s) :=$ max length of a problem set ending at i with spacing s.

Part 1

What do we need in our state?

- The most natural state is $dp(i) :=$ max length of a problem set ending at i.
- But this is not enough information to transition – we need to know the spacing s between problems across states.
- So, this motivates:
  1. **State**: $dp(i, s) :=$ max length of a problem set ending at i with spacing s.
  2. **Transition**:

$$dp(i, s) = 1 + dp(j, s) \text{ for } j \text{ with } d_j = d_i - s$$

Part 1

What do we need in our state?

- The most natural state is $dp(i) :=$ max length of a problem set ending at i.
- But this is not enough information to transition – we need to know the spacing s between problems across states.
- So, this motivates:
  1. **State**: $dp(i, s) :=$ max length of a problem set ending at i with spacing s.
  2. **Transition**:
  $$dp(i, s) = 1 + dp(j, s) \text{ for } j \text{ with } d_j = d_i - s$$
  3. **Base Cases**: $dp(i, s) = 1$ if no j with $d_j = d_i - s$.

Part 1

What do we need in our state?

- The most natural state is $dp(i) :=$ max length of a problem set ending at i.
- But this is not enough information to transition – we need to know the spacing s between problems across states.
- So, this motivates:
  1. **State**: $dp(i, s) :=$ max length of a problem set ending at i with spacing s.
  2. **Transition**:
  $$dp(i, s) = 1 + dp(j, s) \text{ for } j \text{ with } d_j = d_i - s$$
  3. **Base Cases**: $dp(i, s) = 1$ if no j with $d_j = d_i - s$.
- Our answer is simply $\max_{i,s} dp(i, s)$.

Part 1

What do we need in our state?

- The most natural state is $dp(i) :=$ max length of a problem set ending at i.
- But this is not enough information to transition – we need to know the spacing s between problems across states.
- So, this motivates:
  1. **State**: $dp(i, s) :=$ max length of a problem set ending at i with spacing s.
  2. **Transition**:
  $$dp(i, s) = 1 + dp(j, s) \text{ for } j \text{ with } d_j = d_i - s$$
  3. **Base Cases**: $dp(i, s) = 1$ if no j with $d_j = d_i - s$.
- Our answer is simply $\max_{i,s} dp(i, s)$.
- Our time complexity is $O(ND)$.

Part 2

How do we reduce the size of the D state?

Part 2

How do we reduce the size of the D state?

- Are **all** spacings really possible?

Part 2

How do we reduce the size of the D state?

- Are **all** spacings really possible?
- For some i, we should only consider a value of s if there exists some j with $d_i - d_j = s$.

Part 2

How do we reduce the size of the D state?

- Are **all** spacings really possible?
- For some i, we should only consider a value of s if there exists some j with $d_i - d_j = s$.
- We can directly make this a part of our state...

Part 2

How do we reduce the size of the D state?

- Are **all** spacings really possible?
- For some i, we should only consider a value of s if there exists some j with $d_i - d_j = s$.
- We can directly make this a part of our state...
- So, this motivates:

Part 2

How do we reduce the size of the D state?

- Are **all** spacings really possible?
- For some i, we should only consider a value of s if there exists some j with $d_i - d_j = s$.
- We can directly make this a part of our state...
- So, this motivates:
  1. **State**: $dp(i, j) :=$ max length of a problem set with last problem i and second last j.

Part 2

How do we reduce the size of the D state?

- Are **all** spacings really possible?
- For some i, we should only consider a value of s if there exists some j with $d_i - d_j = s$.
- We can directly make this a part of our state...
- So, this motivates:
  1. **State**: $dp(i, j) :=$ max length of a problem set with last problem i and second last j.
  2. **Transition**:

$$dp(i, j) = 1 + dp(j, k) \text{ for } k \text{ with } d_j - d_k = d_i - d_j$$

Part 2

How do we reduce the size of the D state?

- Are **all** spacings really possible?
- For some i, we should only consider a value of s if there exists some j with $d_i - d_j = s$.
- We can directly make this a part of our state...
- So, this motivates:
  1. **State**: $dp(i, j) :=$ max length of a problem set with last problem i and second last j.
  2. **Transition**:
  $$dp(i, j) = 1 + dp(j, k) \text{ for } k \text{ with } d_j - d_k = d_i - d_j$$
  3. **Base Cases**: $dp(i, j) = 2$ if no k with $d_j - d_k = d_i - d_j$.

Part 2

How do we reduce the size of the D state?

- Are **all** spacings really possible?
- For some i, we should only consider a value of s if there exists some j with $d_i - d_j = s$.
- We can directly make this a part of our state...
- So, this motivates:
  1. **State**: $dp(i, j) :=$ max length of a problem set with last problem i and second last j.
  2. **Transition**:
  $$dp(i, j) = 1 + dp(j, k) \text{ for } k \text{ with } d_j - d_k = d_i - d_j$$
  3. **Base Cases**: $dp(i, j) = 2$ if no k with $d_j - d_k = d_i - d_j$.
- Our answer is simply $\max_{i,j} dp(i, j)$.

Part 2

How do we reduce the size of the D state?

- Are **all** spacings really possible?
- For some i, we should only consider a value of s if there exists some j with $d_i - d_j = s$.
- We can directly make this a part of our state...
- So, this motivates:
  1. **State**: $dp(i, j) :=$ max length of a problem set with last problem i and second last j.
  2. **Transition**:
  $$dp(i, j) = 1 + dp(j, k) \text{ for } k \text{ with } d_j - d_k = d_i - d_j$$
  3. **Base Cases**: $dp(i, j) = 2$ if no k with $d_j - d_k = d_i - d_j$.
- Our answer is simply $\max_{i,j} dp(i, j)$.
- Our time complexity is $O(N^2)$.

Outline

1 Housekeeping

2 We have to go deeper

3 On being not a goldfish

4 More dimensions???

5 **Appendix and resources**

Feedback!

Please fill out feedback form so we know how to run future workshops :D

(a) Feedback form

(b) Attendance form

Arc CLUBS SUPPORTED BY Arc INDEPENDENTLY RUN

UNSW CPMSoc